



PDF Download
3703412.3704451.pdf
18 January 2026
Total Citations: 0
Total Downloads: 499

Latest updates: <https://dl.acm.org/doi/10.1145/3703412.3704451>

RESEARCH-ARTICLE

Selective Graph Convolutional Network for Efficient Routing

NIKITA KARTHIKEYAN, Indiana University Bloomington, Bloomington, IN, United States

HAYAGREEV JEYANDRAN

ROHIT VERMA, Intel Corporation, Santa Clara, CA, United States

RAJEEV SHOREY, Indian Institute of Information Technology Surat, Surat, GJ, India

Open Access Support provided by:

Intel Corporation

Indian Institute of Information Technology Surat

Indiana University Bloomington

Published: 08 October 2024

[Citation in BibTeX format](#)

AIMLSystems 2024: The 4th International
Conference on AI-ML Systems
October 8 - 11, 2024
Louisiana, Baton Rouge, USA

Selective Graph Convolutional Network for Efficient Routing

Nikita Karthikeyan
Indiana University, Bloomington
Bloomington, US
k87nikita@gmail.com

Rohit Verma
Intel Labs
Bangalore, IN
rohit1.verma@intel.com

Hayagreev Jeyandran
Vidyashilp Academy
Bangalore, IN
hayagreevj@gmail.com

Rajeev Shorey
IIIT Surat
Surat, IN
rajeevshorey@gmail.com

Abstract

Modern-day networks require highly intelligent and dynamic routing policies that are capable of routing data efficiently over a wide range of changing topology and load conditions. Machine learning frameworks like graph neural networks and multi-agent reinforcement learning have been applied to routing scenarios that deal with heterogeneous graphs. However, these solutions cannot sustain optimality under higher packet loads, because deep Q networks and graph neural networks for a single agent indiscriminately factor in all neighbouring agents during the training phase. This also leads to higher computational costs during training and implementation. In this paper, we propose a Selective Graph Convolutional Network (*Sel_GCN*) model using Multi-Agent Deep Q Networks and Attention to maximize throughput and minimize latency over various packet load conditions. We evaluate our technique on a simulator and compare it to previous implementations of reinforcement learning and graph neural networks. Our results show that *Sel_GCN* achieves 100% packet delivery while reducing the CPU and memory usage by about 50% and 45% respectively.

CCS Concepts

• **Networks** → **Network algorithms**; **Control path algorithms**; **Traffic engineering algorithms**;

Keywords

Reinforcement learning, Graph neural networks, Routing protocols, Attention, Convolution Networks, Deep learning

ACM Reference Format:

Nikita Karthikeyan, Hayagreev Jeyandran, Rohit Verma, and Rajeev Shorey. 2024. Selective Graph Convolutional Network for Efficient Routing. In *4th International Conference on AI-ML Systems (AIMLSystems 2024)*, October 08–11, 2024, Baton Rouge, LA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3703412.3704451>



This work is licensed under a Creative Commons Attribution International 4.0 License.

AIMLSystems 2024, October 08–11, 2024, Baton Rouge, LA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1161-9/24/10
<https://doi.org/10.1145/3703412.3704451>

1 Introduction

In the realm of networking, the efficient routing of data packets is paramount to maintaining robust network performance, reducing latency and maximizing throughput. Packet routing (where we need to define packet forwarding rules to minimize delivery time and maximize packet delivery) is an extensively researched problem in computer networks. Traditional approaches [22, 24], however, struggle to cater to the challenges of dynamic and heterogeneous nature of wireless networks.

With the advent of Artificial Intelligence and Deep Learning, neural networks to solve routing problems. Several works in literature have attempted to apply machine learning to routing problems. Works like You et al. [35] utilize Deep Q Networks (DQN) [20] for dynamic routing decisions. The Multi-Agent Packet Routing (MAPR) model [21] routes packets in a grid, using a Double Deep Q Network (DDQN) [27] at each agent for making routing decisions. Thereby, it is a Multi-Agent Reinforcement Learning (MARL) algorithm.

As graphs are a unique data structure in their own right, using a neural network that was designed to handle them would seem natural. Graph convolutional reinforcement learning [11] uses Reinforcement Learning and Graph Convolutional Networks (GCN) [13] along with an Attention mechanism [28] to solve the packet routing problem. NODE-SELECT [19] has implemented a similar idea and has demonstrated superior performance over baselines, which include Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT).

Although MARL and GCN provide promising directions, several key challenges still remain with the works that have utilized these approaches. First is the significant challenge of scalability plaguing most of these works in literature. Maintaining network information and online learning takes a hit with ever-growing networks, especially in a wireless setup, resulting in either a sub-par model or a high-latency solution. Both of these are highly detrimental to the packet routing decision-making process. Second, the models supporting highly scalable systems do it at a much higher computational and resource cost during training and implementation. Such high costs restrict the approaches to high-end devices and cannot be used for low-resource devices such as UAVs or IoT nodes.

In light of this, we propose a Selective Graph Convolution Network approach, *Sel_GCN*, that allows the nodes to interact and accumulate information from only a select group of neighbours. This restriction on selecting neighbours ensures that the model can

be extended to much larger networks and that resource utilization can be controlled. Our key contributions are as follows;

- Design a packet routing algorithm that uses Graph Convolutional Networks and Multi Head Attention. (Section 4).
- Design a neighbor selection heuristic which restricts the neighbors with which a node can interact. (Section 4.6).
- Design a simulation environment to test *Sel_GCN* in different scenarios (Section 5).

In the next section, we give background of packet routing and state-of-the-art literature (Section 2). Next, we describe the packet routing problem (Section 3) and the design of *Sel_GCN* in detail (Section 4). In the following section, we describe the simulator and evaluate *Sel_GCN* (Section 5). Lastly, we discuss limitations and future directions for *Sel_GCN* (Section 6) before concluding (Section 7).

2 Related Work

Early approaches to the packet routing problem involved likening them to the problem of finding shortest paths, which was resolved using Dijkstra’s [7] or Bellman-Ford’s [2] algorithms, for instance. DB Johnson [12] proposed an algorithm for sparse graphs i.e. graphs where the number of edges is much less than the maximal number determined by the number of nodes.

Contemporary approaches include routing protocols, such as flooding [23], where a recipient sends a packet to all its neighbours except the sender. This may cause congestion, which can be reduced by implementing localised flooding (LSTMflood) [18]. Distance vector multi-case routing (DVMRP) [31] is based on [2] and pruning, a congestion-control algorithm. Models such as Mixed-Integer Linear Programming (MILP) and a metaheuristic algorithm [6] have also been suggested. However, these struggle to perform in high-dimensional action spaces, i.e. in networks with a large number of routers.

Dynamic networks are suitable candidates for applying Reinforcement learning (RL). The uncertainty in decision-making likens them to a control theory problem, where states, actions, etc., can be defined clearly. The Q Learning algorithm [33] uses the Bellman equation to assign and iteratively update a Q value for all state-action pairs. Using these iterative updates, the agent(s) in the system can converge to a policy that maximizes the reward obtained. The Q-routing algorithm [3] uses Q-Learning on agents operating independently. Q-values are indicators of packet latency. However, maintaining these Q-tables at scale is infeasible as they grow exponentially with respect to the number of nodes in the network.

With the advent of Deep Learning, neural networks were used to model the Q value functions. The Deep Q Network (DQN) [20] uses a neural network to estimate the Q value of an action starting from a given state. They are trained over multiple experience tuples (s, a, r, s_{next}) provided by the agent while operating in the environment in real-time.

DQNs, however, are vulnerable to maximizing bias [26], where they overestimate Q-values. Double Q-learning [10] alleviates this by employing two Q functions, interchangeably used to predict the policy or the Q-value. Neural networks are used to model these functions, leading to the Double Deep Q network (DDQN) [27].

The MAPR model [21] follows a decentralised approach in which each agent has its own DDQN and uses information from its immediate neighbours to make routing decisions. The DRL-FIR [17] strategy uses a DQN for packet routing but instead uses information from all the neighbours within two hops of the router to decide the next recipient of the packet.

Extending the foresight of the router beyond its immediate neighbours could be smoothly incorporated by using Graph Neural Networks (GNNs) [25]. In GNNs, the nodes and edges of a graph have learnable representations (feature vectors). Since networks are inherently graphs, GNNs are well suited for modelling them. They are also frequently utilized in modelling chemicals and in social networks [9].

In Graph Convolutional Networks (GCNs) [13], each node uses convolutions to aggregate information from its neighbours. In regular convolutional neural networks (CNNs) [16], a cell condenses information from fixed-size neighbouring regions in each layer. The GCN [13] borrows this concept. In each iteration, information from the immediate neighbouring nodes or edges is accumulated to update the node (or edge) representation. GCNs have been further enhanced by relying on other models such as Auto Encoders as in variational graph auto-encoder (VGAE) [14]. But more recently, attention mechanisms [1] have been successfully incorporated into GCNs.

The Attention mechanism was first introduced for machine translation [1], in which an encoder-decoder model is used. The decoder model focuses (gives attention) on specific parts of the input. The Transformer model [28] exploits this concept and uses Scaled Dot-Product Attention. This can be thought of as using the key to obtain the value for a given query, much like in databases. The encoder of the Transformer generates the values and keys which is used along with the query generated by the decoder to predict the output. Attention mechanisms have also been used to obtain grandmaster level in demanding games like StarCraft II. AlphaStar [30] makes use of a self-attention [5] mechanism, where the query, keys and values are calculated from the same source.

The Transformer model [28] has even been used in Decision Making as demonstrated by the Decision Transformer [4]. It takes the reward, state, and action for the past K time steps $R_1, s_1, a_1, \dots, R_K, s_K, a_K$ and predicts the following action. The Transformer learns the hidden states used to obtain actions by a neural network. The Transformer model is also used in multi-agent scenarios [34] where a Multi-Agent Transformer (MAT) maps observations to actions. A group of agents are assigned a decision order, and each agent chooses its actions based on the actions of its predecessors.

The combination of attention and RL has also been used for routing problems as in the heterogeneous attention-based DRL (HADRL) method [8]. The attention mechanism has also enhanced Graph Neural Networks, like Graph Attention Networks (GATs) [29]. Here, the attention values are used as coefficients to determine each neighbour’s contribution in updating a node’s representation. The heterogeneous graph attention network (HAN) [32] works on heterogeneous graphs, i.e. graphs that have different types of nodes and edges.

Lastly, RL (DQN) [20], Graph Convolutional Networks [13] (2 hops) and multi-head attention have been used in DGN [11] to route packets from source to destination, and have proven to be very

effective. But even Graph Convolutional Networks can be improved by accumulating information from a selective set of neighbours rather than all of its neighbours, as demonstrated by our *Sel_GCN*.

3 Problem Definition

In this section, we first describe the network and the related terminologies and then define the problem in detail.

Table 1: Network

A	X	X	A	A	I	A	A	X	X
A	A	X	A	X	A	A	X	A	A
A	A	A	A	X	A	A	A	A	A
A	A	A	I	A	A	A	A	A	A
A	A	A	X	A	A	A	A	A	I
X	X	A	I	A	A	X	A	X	A
A	A	A	A	A	A	A	X	A	A
A	A	A	A	A	A	A	A	$\mathbb{B}$	A
A	X	A	A	A	X	A	A	A	A
A	X	A	A	A	X	A	A	I	X

3.1 Network architecture

We define an $(N \times M)$ [21] grid, where each grid element can be one of the following:

- Agents (**A**) - Act as routers. Each agent can interact only with its neighbours.
- IoT Nodes (**I**) - Generate packets to be sent to the Base Station.
- Base Stations (**B**) - Receive the packets generated by the IoT Nodes.
- Block (**X**) - An empty cell that can neither send nor receive packets.

The grid is stationary, i.e. the elements of the grid do not change their locations. The edges of the graphs do not have weights. Each node in the grid can have a maximum of eight neighbours.

3.1.1 Packets (P). Let P_i denote a packet in the grid, and let $\mathbb{P}$ denote the set of all packets in the grid at a given instance. Then $P_i = \{\delta_i, \Pi_i\}$, and $\mathbb{P} = \{P_1, P_2, P_3, \dots, P_{|\mathbb{P}|}\}$.

- Time to Live (δ_i) - Number of time steps left until the packet expires. At each passing time step, the time to live is decremented by a single unit.
- Path (Π_i) - A list containing the node ids of the nodes in the path of the packet.

Each new packet generated by the IoT Nodes is given a fixed base TTL, δ_{def} .

3.1.2 Agents (A). Let A_j denote an agent in the grid, and let $\mathbb{A}$ denote the set of all agents in the grid.

Then $A_j = \{node_id_j, \mathbb{L}_j, x_j, \mathbb{Q}_j, agent_recieve_j\}$ and $\mathbb{A} = \{A_1, A_2, A_3, \dots, A_{|\mathbb{A}|}\}$

- $node_id_j$ - Used to identify the node uniquely.

- Neighbours ($\mathbb{L}_j$) - When an instance of the Agent object is created, it receives a list of neighbours that it is allowed to interact with.
- Position ($x_j(r_j, c_j)$) - row and column of the cell containing the Agent in the grid.
- Packet Queue ($\mathbb{Q}_j$) - List of all packets residing with the Agent. Operates on a FIFO principle.
- Number of packets Received ($agent_recieve_j$) - Total number of packets received by the Agent.

An Agent can only transmit one packet per time step. At each time step, δ_i is decremented by one unit for $\forall i$ such that $P_i \in \mathbb{Q}_j$. At each time step t , A_j receives $\mathbb{Q}$ -values from the model and sends the top-most packet in $\mathbb{Q}_j$ to the neighbour with the highest $\mathbb{Q}$ value (all the $\mathbb{Q}$ values lie between 0 and 1) (epsilon-greedy during training, and greedy during evaluation). We define looping as the cycling of packets between neighbours without making progress toward the Base Station. We decrease the $\mathbb{Q}$ values of all $A \in \mathbb{L}_j$ that are present in the packet's path to 0 to discourage this. If the Base Station happens to be the neighbour of any Agent, it directly receives the packet.

3.1.3 IoT Nodes (I). Let I_k denote an IoT Node in the grid, and let $\mathbb{I}$ denote the set of all IoT Nodes in the grid. Then $I_k = \{\lambda, \delta_{def}, x_k, \mathbb{L}_k, \mathbb{Q}_k\}$ and $\mathbb{I} = \{I_1, I_2, I_3, \dots, I_{|\mathbb{I}|}\}$.

- Generation Rate (λ) - Number of packets generated at fixed time intervals.
- Base time to live (δ_{def}) - Initial time to live assigned to each generated packet.
- Position ($x_k(r_k, c_k)$) - row and column of the cell containing the IoT Node in the grid.
- Neighbours ($\mathbb{L}_k$) - It stores the list of neighbours that it is allowed to interact with.
- Packet Queue ($\mathbb{Q}_k$) - List of all packets residing with the Agent. Operates on a FIFO principle.

At fixed time intervals β , each $I_k \in \mathbb{I}$ generates λ packets and transmits them to some $A_j \in \mathbb{L}_k$ such that $\mathbb{Q}_j$ has the smallest length. If the Base Station happens to be the neighbour of any IoT Node, it directly receives the packet. This repeats until the critical time step, β_c , is reached, after which the IoT Nodes stop generating packets.

3.1.4 Base Station ($\mathbb{B}$). Let $\mathbb{B}$ represent the Base Station. There is only one Base Station in the entire $(N \times M)$ grid.

$\mathbb{B} = \{\mathbb{Q}_{\mathbb{B}}, x_{\mathbb{B}}\}$

- Packets Received ($\mathbb{Q}_{\mathbb{B}}$) - List of all the packets P_i received by the Base Station.
- Position ($x_{\mathbb{B}}(r_{\mathbb{B}}, c_{\mathbb{B}})$) - row and column of the cell containing the Base Station in the grid.

The Base Station can only receive packets from its neighbours and it is the final destination of all the packets generated in the network.

3.1.5 Block (X). A block can neither send nor receive packets and is equivalent to an empty cell; hence, a block stores no information. $\mathbb{X}$ represents the set of all blocks in the grid.

3.2 The Packet Routing Problem

Our defined grid is an undirected graph. Let this graph be denoted by $G = (V, E)$, where $V = \{\mathbb{A}, \mathbb{I}, \mathbb{B}, \mathbb{X}\}$ is the set of all the defined node classes, and E is the set of edges between the nodes. $P_i(s_i, d_i)$ represents a packet generated by $s_i \in \mathbb{I}$ and intended to reach $d_i \in \mathbb{B}$. The packet routing problem is defined by the tuple $(G, \mathbb{P})$ such that for each $P_i \in \mathbb{P}$ the algorithm decides Π_i where $\Pi_i^1 = s_i$, $\Pi_i^{|\Pi_i|} = d_i$, and $(\Pi_i^n, \Pi_i^{n+1}) \in E$. Let the total delivery time for P_i be t_i , then the average delivery time for all packets would be calculated as:

$$\mathcal{T} = \sum_i \frac{t_i}{|\mathbb{P}|} \quad (1)$$

Furthermore, since each IoT Node generates λ packets every β time steps until β_c time steps. The total number of packets sent by all the IoT Nodes $\mathbb{I}$ can be calculated as:

$$p = \frac{\beta_c}{\beta} \times \lambda \times |\mathbb{I}| \quad (2)$$

Hence, the received ratio can be calculated as:

$$\phi = \frac{|\mathbb{Q}_{\mathbb{B}}|}{p} \quad (3)$$

The packet routing problem has two main objectives:

- Maximize ϕ , the throughput of the network.
- Minimize $\mathcal{T}$, the latency of the packets.

4 The Selective Graph Convolutional Network Model

The network of Agents, IoT nodes, Base Stations and Blocks are arranged in a grid and stored as nodes of an undirected graph as defined above (§3.1).

We propose *Sel_GCN* to process this graph. It consists of a Graph Convolutional Network [11], where each agent is assigned a feature vector based on its properties and those of its neighbours. This feature vector is first encoded using a neural network. Then, the encoded vector is updated twice by accumulating information from its immediate (one hop) neighbouring nodes. At the end of the process, each node contains information about itself and the neighbours that are within two hops away from it (An x hop neighbour is x edges from the node). The accumulation of information into a single feature vector takes place using a multi-head attention layer.

Finally, the initial encoding and the output of the two multi-head attention layers are all concatenated and fed into a Double Deep Q Network (DQN) to determine Q values for each action. The actions correspond to the neighbor the packet is sent to for each agent.

Algorithm 1 defines the heuristic, which is used to select the neighbours, and Algorithm 2 is used to generate the *Sel_GCN* model. We describe these in detail in the following sub-sections.

4.1 Network

The network is a heterogeneous graph as its nodes belong to four categories (Agents, IoT Nodes, Base Station and Blocks). We focus on the Agents as they are our model's routers. The graph also stores the assigned value of $\kappa \in [0, 7]$. The parameter κ denotes by how much the upper limit on the maximum number of neighbours is

Algorithm 1 Heuristic

```

1: Input: Agents
2: for node in Agents do
3:   Nb=[]
4:   for neigh in node[neighbour] do
5:     if neigh in (Agents or BaseStation) then
6:       Nb.append(neigh)
7:     end if
8:   end for
9:   Nb = sort(Nb, key = d(neigh, B))
10:  Nb = Nb[:8-κ]
11:  node[neighbour] = Nb
12: end for

```

Algorithm 2 *Sel_GCN*

```

1: Input:  $N, M, \kappa, map$ 
2: Output: Sel_GCN Model
3:
4: graph ← Graph( $N, M, \kappa, map$ )
5: graph.select_neighbours()           ▶ Using Algorithm 1
6: buffer ← Dequeue()
7: main_model( $\theta$ ) ← Sel_GCN()
8: target_model( $\theta'$ ) ← Sel_GCN()
9:
10: for ep in  $n_{train}$  do
11:   for step in  $n_{steps}$  do
12:     if (step %  $\beta == 0$ ) and (step <  $\beta_c$ ) then
13:       graph.IoT_send_packets()
14:     end if
15:      $s^{step} \leftarrow$  graph.get_obs()
16:      $Q^{step} \leftarrow$  main_model( $s^{step}; \theta$ )
17:      $a^{step}, r^{step} \leftarrow$  graph.perform_action( $Q^{step}$ )
18:      $s^{step+1} \leftarrow$  graph.get_obs()
19:     buffer.add( $s^{step}, a^{step}, s^{step+1}, r^{step}$ )
20:      $\mathcal{B} \leftarrow$  buffer.get_sample()
21:      $\theta \leftarrow Adam(\theta, \mathcal{B}, lr = \alpha)$ 
22:      $\theta' \leftarrow \tau\theta' + (1 - \tau)\theta$ 
23:   end for
24: end for
25:
26: return main_model( $\theta$ )

```

deducted. Each Agent stores all the packets it receives in a queue and sends them out one by one.

We represent the state (observation) of the graph as O_G , which contains node representations and adjacencies of each node. This is fed to the various components of the model sequentially.

For each Agent, ρ_j is the negative square of the Euclidean distance to the Base Station, $\mathbb{B}$.

$$\rho_j = -((r_j - r_{\mathbb{B}})^2 + (c_j - c_{\mathbb{B}})^2) \quad (4)$$

The feature representation can be defined as *node_vector_j* = $[|Q_j|, \rho_j, \mathbb{L}_j^1(|Q|), \mathbb{L}_j^1(\rho), \dots, \mathbb{L}_j^{8-\kappa}(|Q|), \mathbb{L}_j^{8-\kappa}(\rho)]$. It contains the number of packets and negative distance from the Base Station for that Agent and each of its neighbours.

Given that each agent has $8 - \kappa$ neighbours, the adjacency Adj is defined as a matrix containing $8 - \kappa$ rows of one hot vectors. Each one hot vector is of length $N \times M$, and is activated for the index corresponding to the neighbour's node id.

Then the observation of the graph, O_G is defined as $[node_vector_1, Adj_1, \dots, node_vector_{N \times M}, Adj_{N \times M}]$. This observation is fed into Sel_GCN .

Sel_GCN returns the Q values for each of the Agent's neighbours, which is used to decide where to route the packet.

4.2 Reinforcement Learning

The Reinforcement Learning (RL) framework can be described as making an agent interact with its environment in order to maximise the cumulative reward that it receives from the environment.

Our problem is a multi-agent problem in which each router is treated as an agent that fully cooperates with other routers with the objective defined in §3.2. The network that transmits the packets based on the decision taken by the agent is the environment. A reward of 10 is given for successful packet delivery, whereas an expired packet yields a reward of -50 .

O_G is the state of the environment at any given time. The action selection follows an epsilon greedy strategy during training and a purely greedy strategy during evaluation. An epsilon greedy strategy chooses a random action with epsilon probability and opts for the action with the maximum Q value for the remaining time.

The RL algorithm used is a Deep Q-network (DQN). In RL, a Q-value is assigned to each state-action pair. In DQN, this Q-value function is modelled by a neural network.

4.2.1 Deep Q-Network. The Deep Q Network (DQN) [20] returns the Q-values Q^t for all the possible actions, which the Agents use to route packets, where $Q^t = [Q_1^t, Q_2^t, \dots, Q_{N \times M}^t]$. With the ϵ greedy method utilized during training, we decide the action a^t as follows: The top-most packet $P_i = Q_j^1$ is sent to:

$$a_j^t = \begin{cases} \mathbb{L}_j^n \mid \arg \max_{\xi} Q_j^{t,\xi} = n & \text{with probability } 1 - \epsilon \\ \mathbb{L}_j^n \mid n \sim DUniform(1, |\mathbb{L}_j|) & \text{with probability } \epsilon \end{cases} \quad (5)$$

This allows a mixture of random exploration to be performed by each agent, allowing it to capture the environment better than if it were purely exploiting the environment by always choosing $\mathbb{L}_j$ with the largest action value. Here, $DUniform$ computes the Probability Distribution Function of the uniform distribution.

4.2.2 Training the DQN. To train the DQN, we use two DQNs in practice: the main DQN, Q and the target DQN, Q' . Each experience at time t, exp^t , consists of an array $[s^t, a^t, s^{t+1}, r^t]$ that is added to the buffer. A set of experiences $\mathcal{B}$ are sampled from the buffer at each time step. We can calculate the target Q Value from the Bellman equation as follows:

$$y^n = r^t + \gamma * \arg \max_{a^{t+1}} Q'(s^{t+1}, a^{t+1}) \quad (6)$$

We can also calculate the predicted Q value as follows:

$$y_{pred}^n = Q(s^t, a^t) \quad (7)$$

Hence, the squared-error cost function for all experiences in $\mathcal{B}$ can be calculated as:

$$J_{\mathcal{B}} = \frac{1}{2 \times |\mathcal{B}|} \sum_n (y^n - y_{pred}^n)^2 \quad (8)$$

Any standard optimization algorithm, e.g. SGD or Adam, may be used to adjust the parameters of the Q-value network to minimize $J_{\mathcal{B}}$. We have used Adam with a learning rate $\alpha = 0.001$. The convergence of Adam is stabilized using soft updates [15] governed by an update coefficient τ . With soft updates in place, the parameters of the target DQN, Q' are calculated from the main DQN Q 's parameters as follows:

$$\theta' \leftarrow (1 - \tau)\theta' + \tau\theta \quad (9)$$

4.3 Graph Convolutional Networks

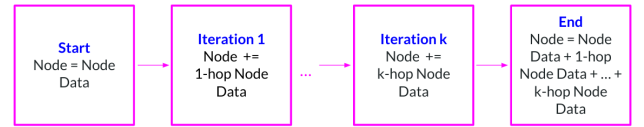


Figure 1: Information Accumulation in GCN

In Graph Convolutional Networks (GCN) [13], each node (or edge) in a graph can have its own feature representation. These representations capture the attributes and properties of the nodes (or edges). The feature representations can be learned or updated by a neural network during the training process.

In GCNs, representations are updated over several iterations. One approach to updating the node representations is to make use of neighbouring nodes, which the DGN model uses. The node representation is updated during each iteration by aggregating information from its immediate neighbours. In this way, each node gradually starts to accumulate information from 1-hop nodes to T-hop nodes after 'T' iterations as shown in Figure 1 (k-hop nodes are defined as nodes that are k edges away from the given node).

This information accumulation allows the node to incorporate information from its surrounding context and capture the relationships between nodes. This enables the GCNs to capture complex patterns and dependencies within the graph structure.

4.4 DGN

The DGN [11] relies on Graph Convolutional Networks with Self Attention to enhance the DQN model.

The node representations are extracted from O_G and sent to the Encoder, which is a two-layer fully connected neural network with ReLU activation. The encoded feature representations are then stacked in feature.

$$\begin{aligned} \text{enc_vector}_j &= \text{Encoder}(\text{node_vector}_j) \\ \text{feature} &= \text{stack}(\text{enc_vector}_j) \end{aligned} \quad (10)$$

The adjacencies of each node are used to collect the encoded feature representations of its neighbours from the tensor, which is then passed to the Graph Convolutional Network (GCN) having

two multi-head attention layers. The first multi-head attention layer uses $enc_neigh_node_j$ to derive a new feature representation of the node $iter_1_node_j$, which is stacked in $relation1$.

$$\begin{aligned} enc_neigh_j &= Adj_j * feature \\ iter_1_j &= Attention_1(enc_neigh_j) \\ relation1 &= stack(iter_1_j) \end{aligned} \quad (11)$$

Then, the same process is repeated for the second iteration.

$$\begin{aligned} iter_1_neigh_j &= Adj_j * relation1 \\ iter_2_j &= Attention_2(iter_1_neigh_j) \\ relation2 &= stack(iter_2_j) \end{aligned} \quad (12)$$

Then the feature representation of each node from enc_vector_j , $relation1$, $relation2$ is fed into the Q Network, where it is concatenated and used to compute the Q values. The Q Network is also a two-layer, fully connected neural network.

4.5 Multi Head Attention

The model we use is a self-attention model [5], [28], which means that the query, key and values are derived from the same source.

Let the source be x . Then, three different fully connected layers are used to transform x into the query (q), key(k) and value (v), respectively. Then, the query is multiplied by the key and passed through a softmax layer. The result is scaled down by $\sqrt{d_k}$ where d_k is the dimension of the key and multiplied by the value to produce the attention.

$$attention = softmax\left(\frac{qk^T}{\sqrt{d_k}}\right)v \quad (13)$$

In Multi-Head Attention [28], the query, key and value are split into several parts. Then the attention is individually computed for each part and concatenated to produce the result. Another fully connected layer processes this result to produce multi-head attention.

4.6 Selection Heuristic

Sel_GCN enhances the performance by allowing each Agent to interact and accumulate information from only a select group of neighbours.

Each Agent can have a maximum of 8 neighbours. The neighbouring nodes are sorted in ascending order based on a specific heuristic chosen as the Euclidean distance to the Base Station in this work. A hyperparameter κ determines the number to be deducted from the maximum upper limit of 8 neighbours. κ can take values from 0 to 7. This means that if an Agent has six neighbours and $\kappa = 4$, then the number of neighbours that the Agent is allowed to interact with is $8 - \kappa = 4$. As the Agent has six neighbours, the last two neighbours that are the furthest away from the Base Station are dropped. The exact number of neighbours to be dropped is found to depend on the map configuration.

By selecting which neighbours are allowed to transmit their information to the node, we prevent the overloading of excess information, thereby allowing the network to exercise better scrutiny.

5 Evaluation

In this section, we first provide the system and simulation details and then evaluate Sel_GCN based on performance and resource utilization aspects.

To ensure reproducibility, we set a seed, which has been assigned a value of 0 for the entire set of evaluation experiments. We use three maps for our experiments that follow the configuration in Table 1. Each map has 75 Agents, 5 IoT Nodes, 1 Base Station and 19 Blocks, but in randomly varied configurations.

For Sel_GCN , the internal dimension of each node representation is 128, and the multi-head attention layer has eight heads. We train the model for one episode where the model updating is performed at each time step after the 100^{th} step, and the model is evaluated at an interval of 100-time steps. After training, the model is evaluated for ten episodes to assess the performance. Table 2 provides the system details where the simulations were run. The details of the simulation parameters are given in Table 3.

5.1 Competing Systems

We compare Sel_GCN with MAPR [21] and the DGN [11] approach. We provide a brief description of the competing systems here.

5.1.1 MAPR: Multi-Agent Packet Routing proposes a decentralised multi-agent reinforcement learning approach where each agent has its own DDQN and uses information from its neighbours to decide the best routing path.

5.1.2 DGN: DGN utilizes a graph convolution network with self-attention to make the routing decisions. The GCN ensures all nodes have a feature representation that captures the attributes and properties concerning itself and the neighbours. However, unlike Sel_GCN , all neighbours are considered in the decision making process.

Table 2: System Details

Specification	Value
Memory	16.0 GiB
Processor	13th Gen Intel® Core™ i9-13900H × 14
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
GPU Memory	6.0 GiB
OS Name	Windows 11 Home

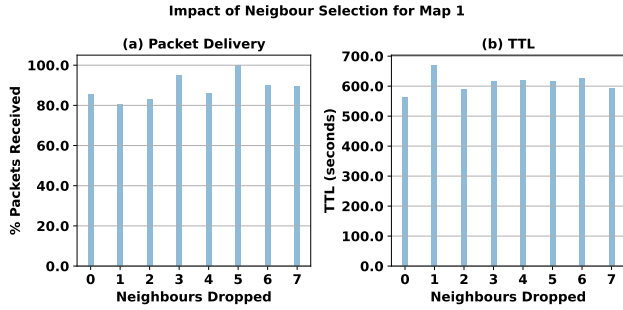
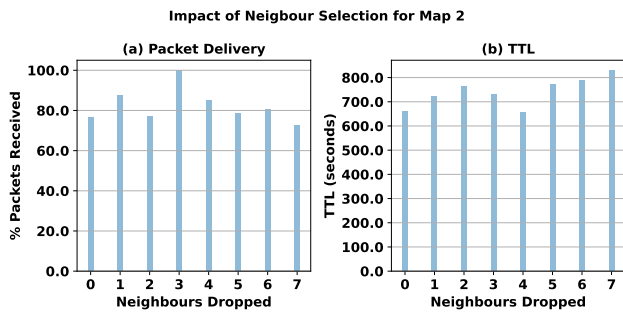
We next evaluate the performance of Sel_GCN on the three different map configurations.

5.2 Packet Delivery

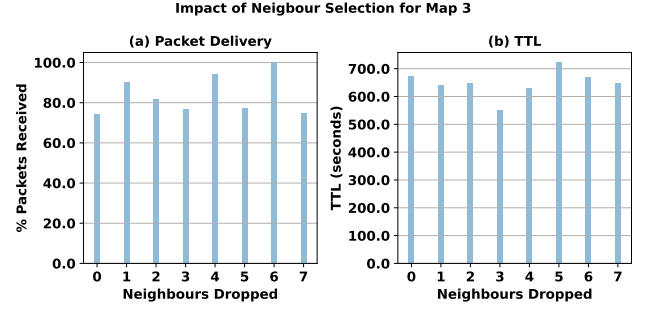
We first assess the performance of Sel_GCN based on the number of packets received by the Base Station on different values of κ , ranging from 0 to 7. The parameter κ denotes by how much the upper limit on the maximum number of packets is deducted. For $\kappa=7$, each agent has only one option for sending packets. Effectively, a value of $\kappa=0$ represents the DGN approach. The packet delivery performance of all three maps is displayed in Figure 2(a), Figure 3(a), and Figure 4(a). As observed in the plot, we get the best performance for different values of κ in the different map configurations.

Table 3: Simulation Parameters

Parameter	Description	Value
N	Number of Rows	10
M	Number of Columns	10
nodes	cells in a grid	100
κ	max neighbors = 8-k	0 to 7
a	action space	8- κ
l	length Obs_i (observation)	$2(8-\kappa)+2$
δ_{def}	Base TTL	1000
λ	IoT Rate	20
β	Gap between packet generation	10
β_c	Last send time step	400
ϵ	Greedy Probability	0.3
γ	Discount Factor	0.98
buffsize	Buffer Capacity	200000
τ	Soft Update Coefficient	0.01
$\mathcal{B}$	Batch Size	8
α	Learning Rate	0.001
n_{train}	Number of train episodes	1
n_{eval}	Number of evaluation episodes	10
$n_{pretrain}$	time steps to fill the buffer	400
n_{steps}	Number time steps per episode	2000

**Figure 2: Packet Delivery and TTL - Map 1****Figure 3: Packet Delivery and TTL - Map 2**

In Map 1, for $\kappa = 5$, the Base Station receives 100% of the packets. It outperforms both $\kappa = 0$ (DGN), which delivers 85% of the packets and $\kappa = 7$, in which the Base Station receives 89% of the packets. A

**Figure 4: Packet Delivery and TTL - Map 3**

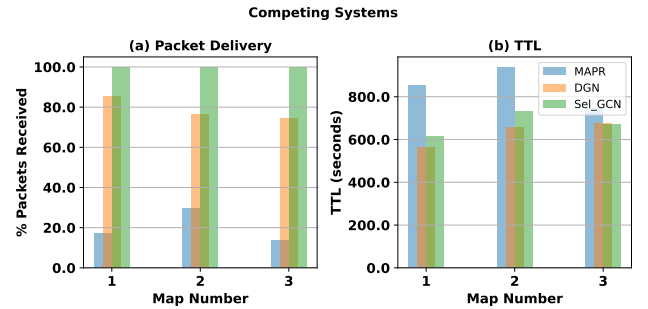
similar trend is seen for Map 2 and Map 3, where the best value of packet delivery (100%) is for $\kappa = 3$ and $\kappa = 6$, respectively.

5.3 TTL of Packets Received

We next assess the performance of *Sel_GC*N based on the mean TTL (time-to-live) of packets received by the Base Station on different values of κ ranging from 0 to 7. We utilize a starting TTL (δ_{def}) of 1000s for all the runs.

In Map 1, $\kappa = 1$ has a mean TTL of 670s out of 1000s (Figure 2(b)). It outperforms both $\kappa = 0$ (DGN), which has a mean TTL of 563s out of 1000s and $\kappa = 7$, which has a mean TTL of 593s out of 1000s. Again similar trends are seen in Map 2 (Figure 3(b)) and Map 3 (Figure 4(b)) where selecting a subset of nodes provides the best results.

It should be noted that the best result for packet delivery and TTL is not obtained for the same value of κ . However, based on the requirements, an optimization strategy could be devised that either gives preference to packet delivery, TTL, or a trade-off strategy that optimizes both.

**Figure 5: Packet Delivery and TTL Comparison**

5.4 Competing Systems: Packet Delivery

Next, we compare the performance of the three competing models based on the percentage of packets received by the Base Station. In these results, we have given preference to packet delivery, hence we use $\kappa = (5, 3, 6)$ for (Map 1, Map 2, and Map 3).

Figure 5(a) shows the packet delivery percentage for the three systems evaluated over the three map configurations. As MAPR

is designed to cater to smaller networks, it performs poorly on a 10x10 grid with the best packet delivery of $\approx 30\%$ in Map 2's configuration. DGN improves considerably on MAPR with $\approx 85\%$ delivery percentage owing to the GCN and attention mechanism. With *Sel_GCN*, this gain is further boosted to a 100% delivery as the nodes need to work on only the relevant information, thus making better decisions. The neural network for *Sel_GCN* is not overloaded with the information from all the neighbours but rather a select few neighbours. Therefore, it can process the information more effectively and discern which neighbour to transmit the packet to.

5.5 Competing Systems: TTL of Packets Received

Next, we compare the performance of the three competing systems based on the mean TTL (time-to-live) of packets received by the Base Station. As we have given preference to packet delivery, hence we use $\kappa = (5, 3, 6)$ for (Map 1, Map 2, and Map 3).

Figure 5(b) shows the TTL for the three systems evaluated over the three map configurations. Although MAPR has the best TTL values, this is because fewer packets are being delivered. But for DGN and *Sel_GCN*, most packets make it to the base station, even the slower packets, reducing the mean TTL of the packets received.

5.6 Competing Systems: Resource Utilization

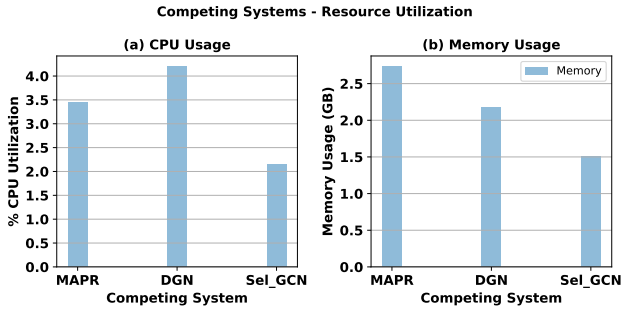


Figure 6: Resource Utilization Comparison

Lastly, we compare the resource utilization of the three competing systems. We compare CPU and system memory utilization. For DGN and *Sel_GCN*, we run the experiments on the map configuration Map 1. We use $\kappa = 5$ for *Sel_GCN*, the best performing κ -value on Map 1 in terms of packet delivery. All metrics are measured every 100 steps and averaged over ten episodes of 2000 steps each. For MAPR, we used the 4x3 map provided by the authors as MAPR is not scalable to operate on a 10x10 grid and performs very poorly, invalidating any comparison. All evaluation is done on a GPU via PyTorch's CUDA framework.

As is evident from Figure 6(a), *Sel_GCN* utilizes the minimum amount of CPU with $\approx 50\%$ improvement compared to DGN and 43% improvement compared to MAPR. Similarly, *Sel_GCN* shows a reduction by 45% compared to MAPR and 31% compared to DGN (Figure 6(b)). Both resource gains for CPU and Memory are because *Sel_GCN* only works with a subset of neighbours in the network. Omitting the farthest five neighbours at each agent allows packets

in *Sel_GCN* to pass through fewer nodes, getting routed around the network fewer times before reaching the base station.

6 Discussion

In this section, we discuss a few limitations that would further improve the work.

6.1 Agent Mobility

We plan on incorporating mobility into the network, which would liken our agents to UAVs and IoTs and Base stations to terrestrial stations sending and receiving data to and from the UAVs, respectively. However, a lightweight model without large training complexity must be devised since UAVs are often constrained devices with limited processing power. The observation of the graph should also factor in additional parameters specific to UAV networks, such as the transmission range and battery percentage, for use in path planning. Such observations will prevent any UAV from moving out of range, failing to operate, and succumbing to an empty battery. If a UAV is about to run out of power, it will have to transmit all of its data to another nearby UAV and land to charge itself.

6.2 Neural Network Selector

In our work, we manually select the value of the κ . We plan to implement a neural network that automatically determines how many neighbours to drop for each agent. This automation would also allow for different κ values for each neighbour, increasing flexibility. We hope to see if it improves the network's performance in the same aspects of latency and throughput.

6.3 Additional Heuristics

In our work, we have chosen to drop κ neighbours based on the negative Euclidean distance to the Base Station since it was a simple and intuitive heuristic to implement. There are multiple other mechanisms that could determine which neighbours to drop, such as the Manhattan or Minkowski distance. These could affect our choice of κ values for optimal performance.

7 Conclusion

Neural network-based solutions have effectively addressed the packet routing problem in dynamic networks. However, the reliance on neural networks also results in higher resource requirements. In this paper, we have proposed *Sel_GCN*, which utilizes a node selection strategy to lower the load on the system by decreasing the state space of the training models. Some key takeaways from our experiments are (i) *Sel_GCN* outperforms other existing systems by achieving 100% packet delivery, thus highlighting that we do not require information from all neighbouring nodes for making the best routing decisions, (ii) decreasing the number of nodes that *Sel_GCN* works with results in a scalable solution that could support larger networks than other solutions like MAPR, (iii) the node selection approach also ensures that *Sel_GCN* requires lesser resources than the competing systems (lowers the CPU and memory usage by about 50% and 45% respectively).

References

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473* (2014).
- [2] Richard Bellman. 1958. On a routing problem. *Quarterly of applied mathematics* 16, 1 (1958), 87–90.
- [3] Justin Boyan and Michael Littman. 1993. Packet Routing in Dynamically Changing Networks: A Reinforcement Learning Approach. In *Advances in Neural Information Processing Systems*, J. Cowan, G. Tesauro, and J. Alspector (Eds.), Vol. 6. Morgan-Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1993/file/4ea06fbc83cdd0a06020c35d50e1e89a-Paper.pdf
- [4] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision Transformer: Reinforcement Learning via Sequence Modeling. *CoRR abs/2106.01345* (2021). [arXiv:2106.01345](https://arxiv.org/abs/2106.01345)
- [5] Jianpeng Cheng, Li Dong, and Mirella Lapata. 2016. Long Short-Term Memory-Networks for Machine Reading. *CoRR abs/1601.06733* (2016). [arXiv:1601.06733](https://arxiv.org/abs/1601.06733)
- [6] Bruno N. Coelho, Vitor N. Coelho, Igor M. Coelho, Luiz S. Ochi, Roozbeh Haghnazar K., Demetrius Zuidema, Milton S.F. Lima, and Adilson R. da Costa. 2017. A multi-objective green UAV routing problem. *Computers and Operations Research* 88 (2017), 306–315. <https://doi.org/10.1016/j.cor.2017.04.011>
- [7] E. W. Dijkstra. 1959. A note on two problems in connexion with graphs. *Numer. Math.* 1, 1 (dec 1959), 269–271. <https://doi.org/10.1007/BF01386390>
- [8] Mingfeng Fan, Yaoxin Wu, Tianjun Liao, Zhiguang Cao, Hongliang Guo, Guillaume Sartoretti, and Guohua Wu. 2023. Deep Reinforcement Learning for UAV Routing in the Presence of Multiple Charging Stations. *IEEE Transactions on Vehicular Technology* 72, 5 (2023), 5732–5746. <https://doi.org/10.1109/TVT.2022.3232607>
- [9] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation Learning on Graphs: Methods and Applications. *CoRR abs/1709.05584* (2017). [arXiv:1709.05584](https://arxiv.org/abs/1709.05584)
- [10] Hado Hasselt. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta (Eds.), Vol. 23. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fcd301b442654dd8c23b3fc9-Paper.pdf
- [11] Jiechuan Jiang, Chen Dun, and Zongqing Lu. 2018. Graph Convolutional Reinforcement Learning for Multi-Agent Cooperation. *CoRR abs/1810.09202* (2018). [arXiv:1810.09202](https://arxiv.org/abs/1810.09202)
- [12] Donald B. Johnson. 1977. Efficient Algorithms for Shortest Paths in Sparse Networks. *J. ACM* 24, 1 (jan 1977), 1–13. <https://doi.org/10.1145/321992.321993>
- [13] Thomas N. Kipf and Max Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. *CoRR abs/1609.02907* (2016). [arXiv:1609.02907](https://arxiv.org/abs/1609.02907)
- [14] Thomas N. Kipf and Max Welling. 2016. Variational Graph Auto-Encoders. [arXiv:1611.07308](https://arxiv.org/abs/1611.07308) [stat.ML]
- [15] Taisuke Kobayashi and Wendyam Eric Lionel Ilboudo. 2020. t-Soft Update of Target Network for Deep Reinforcement Learning. *CoRR abs/2008.10861* (2020). [arXiv:2008.10861](https://arxiv.org/abs/2008.10861)
- [16] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. <https://doi.org/10.1109/5.726791>
- [17] Deping Lin, Tao Peng, Peiliang Zuo, and Wenbo Wang. 2022. Deep-Reinforcement-Learning-Based Intelligent Routing Strategy for FANETs. *Symmetry* 14, 9 (2022). <https://doi.org/10.3390/sym14091787>
- [18] Justin Lipman, Paul Boustead, and Joe Chicharo. 2005. *Localised Minimum Spanning Tree Flooding in Ad-Hoc Networks*. Springer US, Boston, MA, 19–37. https://doi.org/10.1007/0-387-22792-X_2
- [19] Steph-Yves M. Louis, Alireza Nasiri, Fatima J. Rolland, Cameron Mitro, and Jianjun Hu. 2021. NODE-SELECT: A Graph Neural Network Based On A Selective Propagation Technique. *CoRR abs/2102.08588* (2021). [arXiv:2102.08588](https://arxiv.org/abs/2102.08588)
- [20] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. 2013. Playing Atari with Deep Reinforcement Learning. *CoRR abs/1312.5602* (2013). [arXiv:1312.5602](https://arxiv.org/abs/1312.5602)
- [21] Aniket Modi, Rishi Shah, Krishnanshu Jain, Rohit Verma, Rajeev Shorey, and Huzar Saran. 2023. Multi-Agent Packet Routing (MAPR): Co-Operative Packet Routing Algorithm with Multi-Agent Reinforcement Learning. In *2023 15th International Conference on COMMunication Systems and NETWORKS (COMSNETS)*, 722–730. <https://doi.org/10.1109/COMSNETS56262.2023.10041337>
- [22] Charles Perkins, Elizabeth Belding-Royer, and Samir Das. 2003. RFC3561: Ad hoc on-demand distance vector (AODV) routing.
- [23] H.K. Pung, J. Song, and L. Jacob. 1999. Fast and efficient flooding based QoS routing algorithm. In *Proceedings Eight International Conference on Computer Communications and Networks (Cat. No.99EX370)*, 298–303. <https://doi.org/10.1109/ICCCN.1999.805534>
- [24] Yakov Rekhter, Tony Li, and Susan Hares. 2006. *A border gateway protocol 4 (BGP-4)*. Technical Report.
- [25] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Transactions on Neural Networks* 20, 1 (2009), 61–80. <https://doi.org/10.1109/TNN.2008.2005605>
- [26] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.
- [27] Hado van Hasselt, Arthur Guez, and David Silver. 2015. Deep Reinforcement Learning with Double Q-learning. *CoRR abs/1509.06461* (2015). [arXiv:1509.06461](https://arxiv.org/abs/1509.06461)
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *CoRR abs/1706.03762* (2017). [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- [29] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [30] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 7782 (2019), 350–354.
- [31] David Waitzman, Craig Partridge, and Stephen E Deering. 1988. *Distance vector multicast routing protocol*. Technical Report.
- [32] Xiao Wang, Houye Ji, Chuan Shi, Peng Cui, Philip S. Yu, and Yanfang Ye. 2019. Heterogeneous Graph Attention Network. *CoRR abs/1903.07293* (2019). [arXiv:1903.07293](https://arxiv.org/abs/1903.07293)
- [33] Christopher John Cornish Hellaby Watkins. 1989. Learning from delayed rewards. (1989).
- [34] Muning Wen, Jakub Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. 2022. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems* 35 (2022), 16509–16521.
- [35] Xinyu You, Xuanjie Li, Yuedong Xu, Hui Feng, and Jin Zhao. 2019. Toward Packet Routing with Fully-distributed Multi-agent Deep Reinforcement Learning. *CoRR abs/1905.03494* (2019). [arXiv:1905.03494](https://arxiv.org/abs/1905.03494)